The AVR is a line of popular 8-bit RISC microcontrollers from Atmel. Here is why I like them.

Feel free to email me if you have a question on any of the stuff below but note that I do all of my AVR work exclusively in Linux. If you have a question on how to make AVR programming work in microsoft windows you are probably much better off heading on over to Avr freaks to get help.

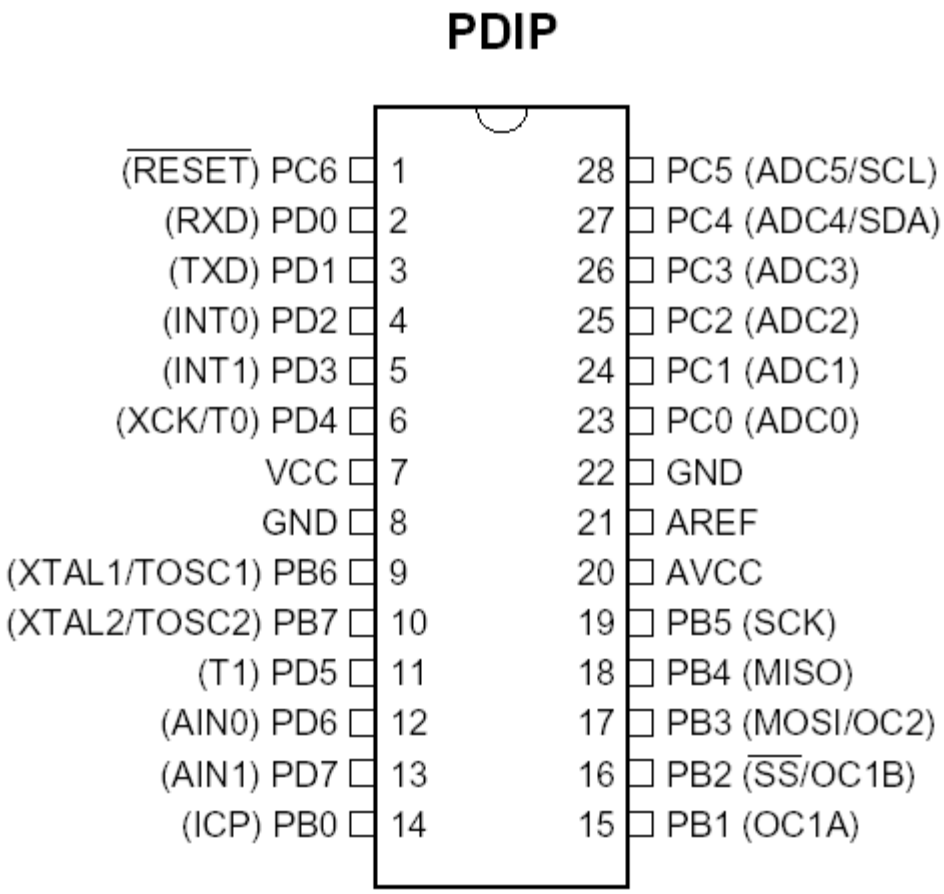
A good choice of controller to start with is the mega8. It inexpensive and quite capable. Why that particular processor? Well, if I want to program it in C using gcc the processor needs to have a ram-based stack. The smallest AVR that is supported by GCC is the tiny26. I also like to stick with DIP packages when I can as that makes plugging them into breadboards easier. The mega32 is the largest AVR that comes in DIP form. The mega8 is right in between.

The Western Canadian Robot Society that I am a part of has had a number of its members settle on using the AVR chips as a microcontroller base. At our recent AVR FreakFest 13 of us got together to make cables, discuss programming the AVR (here in action aboard my home-made experimenter's board and get to meet my super-friendly cat Fluffy. . Here is a less scary picture. Thanks to Grant from Solarbotics for taking the pics.

The "l" part of the name signifies that it is the low power model and will work from 2.7 - 5.5V power supplies and up to 8 MHz clock speeds. The other version needs 4.5 - 5.5V but will work up to 16 MHz. There are rumours that the slower versions will likely also work up to 16 MHz.

A few highlights of its capabilities:
- 23 I/O lines
- In System Programmable (i.e cheap and easy for us to program)
- 1K RAM
- 8K FLASH PROM
- 512 bytes EEPROM
- 2 8 bit counter/timers
- 1 16 bit counter/timer
- three PWM channels (one 8-bit, two 10/8-bit) Great for that omnidirectional robot I want to build somedayrealsoonnow.
- programmable serial USART
- master/slave SPI interface
- programmable watchdog timer
- on chip analog comparator
- 6 channel ADC converter (4 10-bit, 2 8-bit)
- built-in RC clock if you can't be bothered with crystals/resonators
- programmable pullup resistors on input pins
- and a whole schwack of other cool and useful stuff

Here's a pinout:



For the system clock, you can use either:

- built-in RC clock which will run up to 8MHz (cheapest, least accurate, uses no I/O pins),
- attach a ceramic resonator (needs 2 caps and resonator or just 3-pin resonator, more accurate than internal RC clock, uses 2 I/O pins)
- attach a crystal (same as resonator above but can be even more accurate)
- attach an external oscillator (needs oscillator and no caps, as accurate as crystal, uses only one I/O pin, eats power).

From the factory, it is automagically setup to use it's internal RC oscillator at 1 MHz. If you want to change the clock speed or where the clock comes from you can adjust it by setting "fuses" with the programmer software. They can't be set from within the program you download to the AVR.

The IO pins are lumped into three groups, B, C and D and are labeled Dxn where X is the group and n is the number. i.e. PB1. You can even use the !reset pin (pin 1) as I/O but I recommend against that if you are programming it with ISP. There is no "A" port. Don't ask me why.

The cable I use (the schematic is on the AVR resources page and on Quentin's CD) is the "DAPA" style programmer. I use UISP (free software) to drive it and I have confirmed it works in both Windows (at least 2000 and XP) and Linux. Don't omit the resistors. Although it says elsewhere (and I've echoed it) we don't need them if we aren't using those pins for I/O I put them in anyways as insurance. If I screw up and short across them they will limit the current and help prevent blowing something up. Replacing a motherboard to fix the parallel port would be quite annoying.

We have also confirmed that the sp12 programming cable works in linux and windows. I would strongly recommend that you add 220-470 ohm resistors for the reset and MISO lines. They aren't really necessary for it to work but it can help protect your parallel port if a booboo is made. For the cables we made as a mob during our 2003 AVR freakfest we omitted the power connections completely. I've also found that I have not had a problem omitting the 100 ohm resisttor and 22pf capacitor for the SCK line. If you decide to add that remember they are to be mounted near the microcontroller.

I would recommend that you build the SP12 cable because avrdude, the program that's used to actually upload the code to your AVR understands the SP12. Avrdude is part of WinAvr, the freely available development environment available from AVR Freaks .

For the other end we want with the pinout that's used on the STK500 and AVR butterfly.

Seen from above the header on the board the cable plugs into

ISP header

blue	MISO	+voltage	red
purple	SCK	MOSI	grey
white	RST	GND	black

The colours are just what I use on my standard connectors

I used to always leave the +voltage wire unconnected but since getting the Atmel AVRISP MK II I pull it up to +voltage with a 10K resistor. This programmer won't program the board unless it can get what voltage to program at. This is a great programmer module but I sure wish they would have documented this fact in the programmer rather than in Avr Studio (which I do not use).

So, now on to some programming.

Here is the old flashing light program. Hook up an LED with a 220 - 330 ohm resistor in series to pin 15 and the other end to +5. Add power. You don't need a crystal if you haven't changed the clock fuses. I updated this program in April 2007 to try and keep up to date with the changes going in avr-libc.

```
#include < avr/io.h >              // snag stuff we need
// remove the spaces around the "avr/io" in the line above.  html won't
// let me use the less-than and greater-than symbols the way I want.
// Yeah, I know, there is probably way of doing this but I can't be
// bothered to figure it out.

// the latest versions of avr-libc have deprecated the sbi and cbi macros.
// Well, I won't give them up so I now define them in my program
#define sbi(sfr, bit) (_SFR_BYTE(sfr) |= _BV(bit))
#define cbi(sfr, bit) (_SFR_BYTE(sfr) &= ~_BV(bit))

int main(void)                      // always have to have a place to start
{
  long x;                           // couple of 32 bit variables*

  DDRB = _BV(DDB1);                 // set PB1 (pin 15) as output
  while (1)                         // do it forever
  {
    cbi(PORTB, PB1);                // set PB1 to "low" (LED ON)
```

```
    for (x=0; x < 100000; x++)
      asm volatile ("nop" ::);
    sbi(PORTB, PB1);                // set PB1 to "HIGH" (LED OFF)
    for (x=0; x < 100000; x++)
      asm volatile ("nop" ::);
  }
}
```

The wait loops have the asm volatile ("nop" ::); just to have something to do since I've seen some of the better compilers might optimize empty loops out of existance.

Using a higher level language like this makes our job so much easier. For example, in the program above I used the variable, x of the type long. That's a 32 bit integer. The mega8l is an 8 bit processor. To do 32 bit arithmetic in assembler language on an 8-bit processor is a fair amount of work. Here, we can toss them around at will and not have to worry about the details.

Below are some links that describe some of the ways I program the AVR microcontrollers to do various things:

TIMERS ROCK!

Interrupts and how the heck to deal with them

Dealing with humans and their annoying tendency to press buttons New august 20, 2004.

Using the USART to interface to an LCD display New august 23, 2004

I suffer the pain of the gotchas so you don't have to. New May 23, 2007.

PWM PART DEUX: How to do get forward, reverse, brake, coast and locked anti-phase with just two I/O pins and switch between them entirely in software. New June 11, 2007.

Debugging I2C tool: using another mega to sample I2C traffic and graphically display it on your computer using java. New Sept 14, 2007

SORRY, the links below don't go anywhere yet. Under construction.

Analog to digital conversion for smarties

Motor control part trois: getting feedback or "WHY IS THIS STUPID ROBOT NOT GOING WHERE THE HECK I AM TELLING IT TO? GGRRRRRRRRRRR"

Interfacing the AVR to the Analog Devices ADXRS300 angular rate sensor.

Putting it all together. Climber's Omnihex, an omnidirectional robot

---

to email Craig send to climber at shaw.ca (replace at with @ and remove spaces).
Return to Craig's Electronics page.
Check out what's new on Craig's pages.

Return to Craig's main page.

Last modified: July 5, 2004 - Whaddaya mean there's no coke?!?!?!